Table of Contents

Introduction **2**

Input Validation from the Server Side **2**

Verify Input Validation from the Server Side **5**

Input Validation from the Client Side via jQuery **6**

Demo: Input validation

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT.

Introduction

This demo will explore input validation both on the server side and the client side, enhancing the robustness and usability of our web application. The focus is improving user experience by ensuring data integrity and providing informative feedback to users.

Input Validation from the Server Side

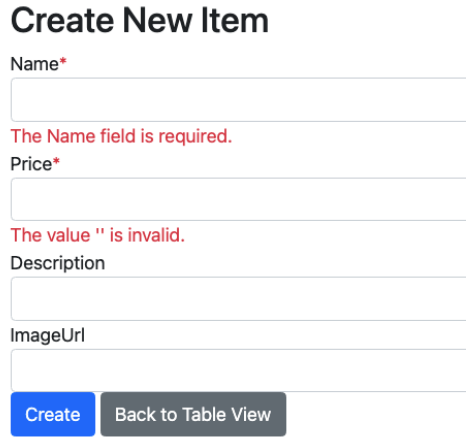
- Incorporate data annotations in data models, e.g., `Models/Item.cs` class to enforce validation rules such as required fields, data types, and maximum length constraints.
- Update the View files, e.g., `Views/Item/Create.cshtml` view to display validation summaries and error messages effectively, ensuring users are informed of any input errors.

Input Validation from the Client Side

- Client-side validation can improve performance and user experience by providing immediate feedback.
- Use jQuery validation libraries (`jquery-validate` and `jquery-validation-unobtrusive`) to perform input checks on the client side before data is sent to the server. This will prevent invalid data from being submitted and reduce server load.

Input Validation from the Server Side

In the previous demos, we already had limited version of input validation (see ShopDatabase-2-CRUD).
When some properties of a class are required, but the user does not fill in the form, the form will pop up and remind that some fields are required. (see Name as below)
When the user fills a data with a different datatype than the required one, the form will also pop up and remind the value is invalid. (see Price as below)



This time, we are going to introduce some more advanced input validation.
We take the Models/Item.cs as example, and add several annotations to it:

Item.cs

```
election
1   using System.ComponentModel.DataAnnotations;
2   using Microsoft.AspNetCore.Mvc.ModelBinding;
3
4   namespace MyShop.Models
5   {
6       public class Item
7       {
8           public int ItemId { get; set; }
9
10          [RegularExpression(@"[0-9a-zA-ZæøåÆØÅ. \-]{2,20}", ErrorMessage = "The Name must be numbers
              or letters and between 2 to 20 characters.")]
11          [Display(Name = "Item name")]
12          public string Name { get; set; } = string.Empty;
13
14          [Range(0.01, double.MaxValue, ErrorMessage = "The Price must be greater than 0.")]
15          public decimal Price { get; set; }
16
17          [StringLength(200)]
18          public string? Description { get; set; }
19
20          public string? ImageUrl { get; set; }
21          // navigation property
22          public virtual List<OrderItem>? OrderItems { get; set; }
23      }
24  }
25
26
```

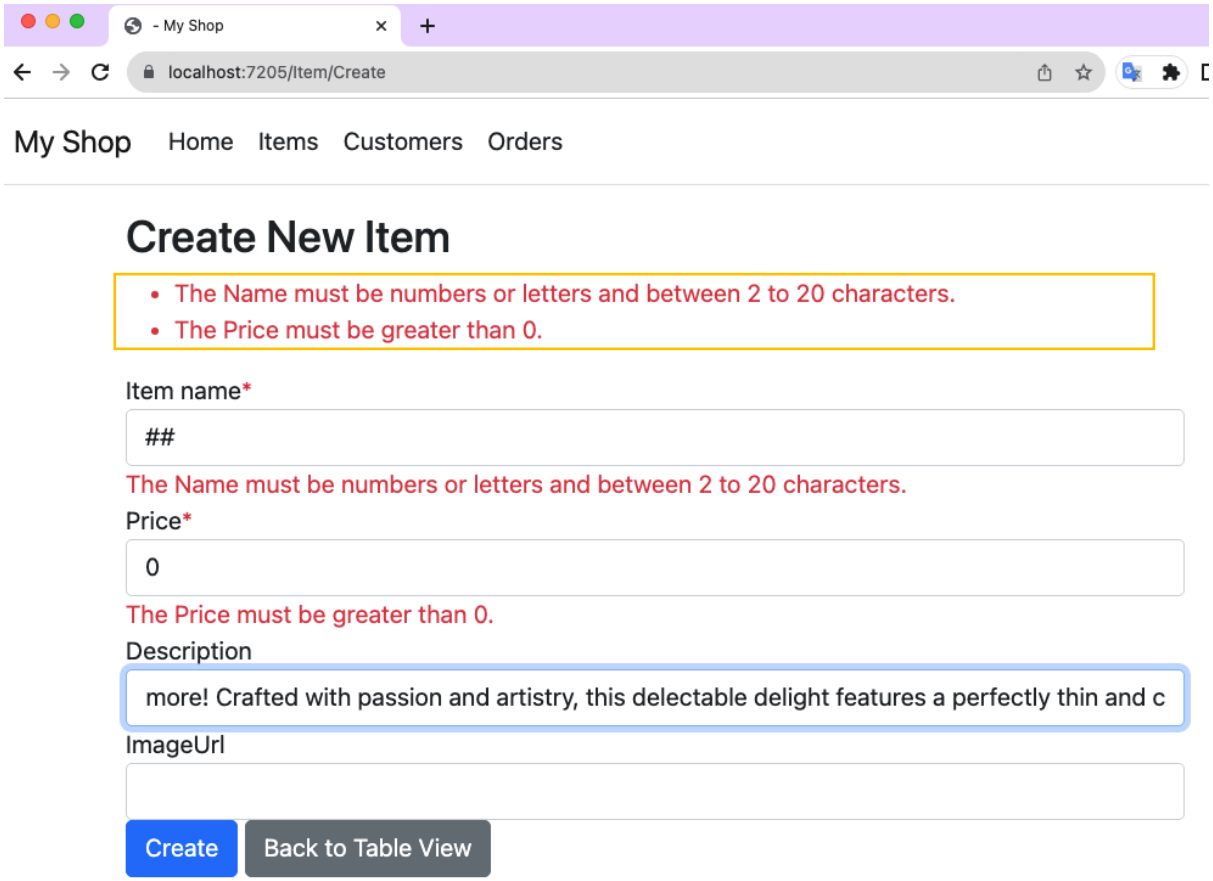
Line1: imports the dependency for introducing input validations
Line10 – Line18, various validations requirements.
Line11: changing the display name in the form.
Line17: specify the allowed maximal length of characters.
In the Views/Item/Create.cshtml, we can also introduce a validation summary (Line5)

Create.cshtml

Views > Item > Create.cshtml

```
1   @model Item
2
3   <h2>Create New Item</h2>
4
5   <form asp-action="Create">
6       <div asp-validation-summary="All" class="text-danger"></div>
7       <div class="form-group">
8           <label asp-for="Name"></label><span class="text-danger">*</span>
9           <input asp-for="Name" class="form-control" />
10          <span asp-validation-for="Name" class="text-danger"></span>
11      </div>
12      <div class="form-group">
13          <label asp-for="Price"></label><span class="text-danger">*</span>
14          <input asp-for="Price" class="form-control" />
15          <span asp-validation-for="Price" class="text-danger"></span>
16      </div>
17      <div class="form-group">
18          <label asp-for="Description"></label>
19          <input asp-for="Description" class="form-control" />
20          <span asp-validation-for="Description" class="text-danger"></span>
21      </div>
22      <div class="form-group">
23          <label asp-for="ImageUrl"></label>
24          <input asp-for="ImageUrl" class="form-control" />
25          <span asp-validation-for="ImageUrl" class="text-danger"></span>
26      </div>
27      <button type="submit" class="btn btn-primary">Create</button>
28      <a asp-action="Table" class="btn btn-secondary">Back to Table View</a>
29  </form>
```

The validation information is shown if the user does not follow the requirements.
The description text is cut because it cannot exceed 200 characters.
Above the form, the summary of the validation messages is also shown.
Run the project, and you can test the input validation:


- My Shop
localhost:7205/Item/Create
My Shop
Home
Items
Customers
Orders
Create New Item
The Name must be numbers or letters and between 2 to 20 characters.
The Price must be greater than 0.
Item name*
##
The Name must be numbers or letters and between 2 to 20 characters.
Price*
0
The Price must be greater than 0.
Description
more! Crafted with passion and artistry, this delectable delight features a perfectly thin and c
ImageUrl
Create
Back to Table View

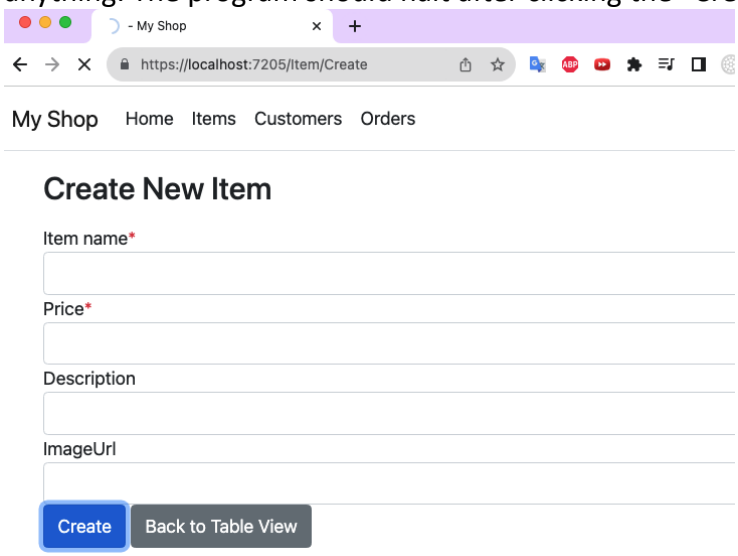
Verify Input Validation from the Server Side

In the previous section, the input validation is done from the server side. This is not always necessary, because the server is busy, and this simple input validation can also be done from the client side.

To verify that it is done from the server side, we create a breakpoint at the first line of [HttpPost]Create method in Controllers/ItemController.cs.

```
60      [HttpPost]
61      public async Task<IActionResult> Create(Item item)
62      {
63          if (ModelState.IsValid)
64          {
65              bool returnOk = await _itemRepository.Create(item);
66              if (returnOk)
67                  return RedirectToAction(nameof(Table));
68          }
69          _logger.LogWarning("[ItemController] Item creation failed {@item}", item);
70          return View(item);
71      }
```

Then we start the project with debugging mode, and try to create an Item without inputting anything. The program should halt after clicking the “Create” button



Through the arrow we can see where it is halt.

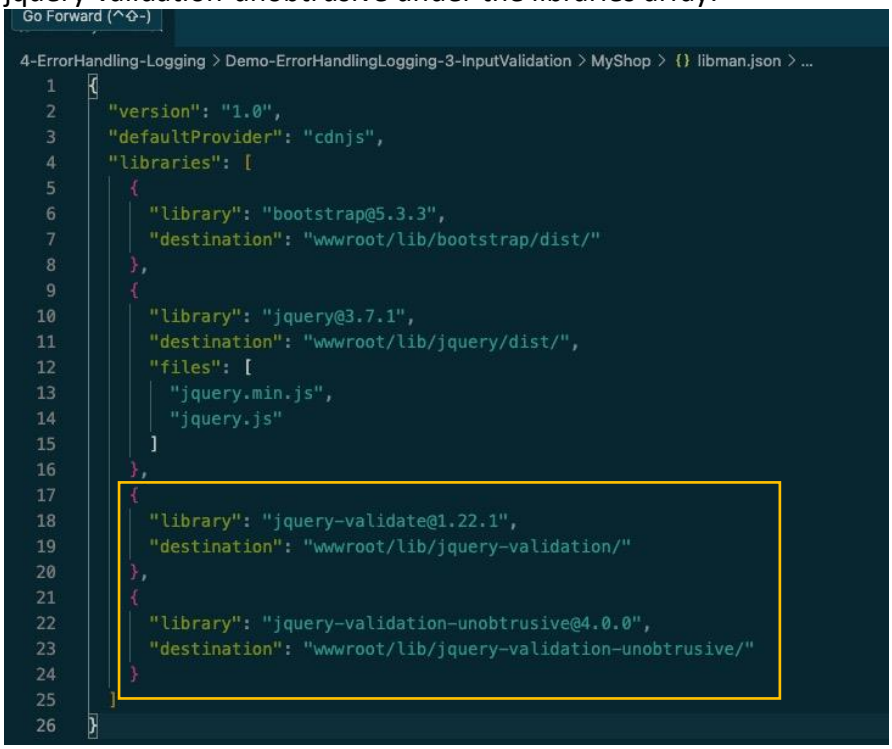
```
60      [HttpPost]
61      public async Task<IActionResult> Create(Item item)
62      {
63          if (ModelState.IsValid)
64          {
65              bool returnOk = await _itemRepository.Create(item);
66              if (returnOk)
67                  return RedirectToAction(nameof(Table));
68          }
69          _logger.LogWarning("[ItemController] Item creation failed {@item}", item);
70          return View(item);
71      }
```

Input Validation from the Client Side via jQuery

We can use LibMan (Library Manager) to pull client-side validation libraries and automatically enable client-side validation based on our server-side data annotations.

Step 1: Update libman.json

Open libman.json in the root of your project and add the entries for jquery-validation and jquery-validation-unobtrusive under the libraries array:



You can use https://cdnjs.com/ to check the version number.

Alternatively, you can install a local libman tool

```
dotnet tool install Microsoft.Web.LibraryManager.Cli
```

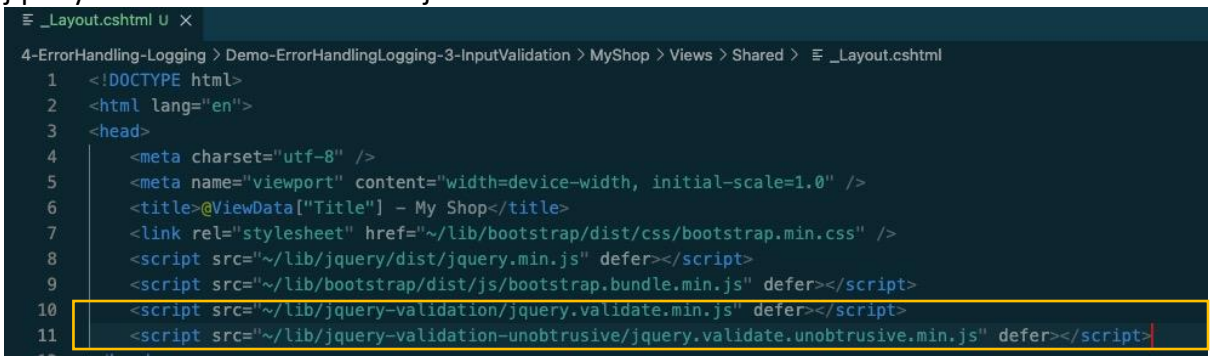
This registers LibMan inside dotnet-tools.json and downloads the binary locally for your project.

Then you can use the below command to manage the libraries.

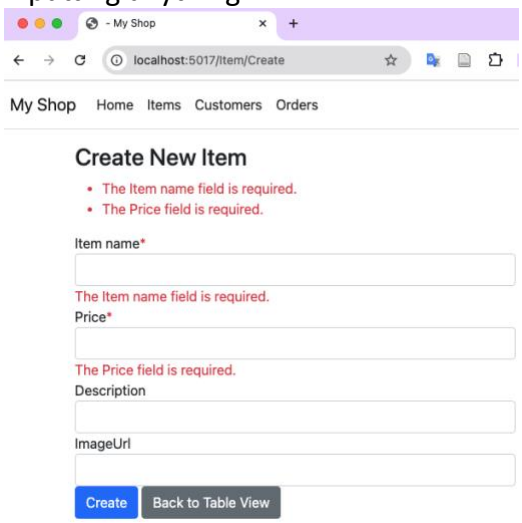
```
dotnet libman install [packagename]
dotnet libman update [packagename]
```

Step 2: Include the scripts in Views/Shared/_Layout.cshtml

Ensure jquery is loaded **first**, followed by jquery.validate.min.js, and finally jquery.validate.unobtrusive.min.js in the <head> with defer:



Now run the project in debugging mode (F5) again, and try to create an Item without inputting anything:



The application does not halt anymore at the breakpoint. This means the Http request is not sent to the server yet, the input validation is already handled from the client side.